

Online

Transformação Digital no Setor Público

Aula 6 — Abstração e Padronização: os motores da evolução tecnológica e do governo digital

Aula 6

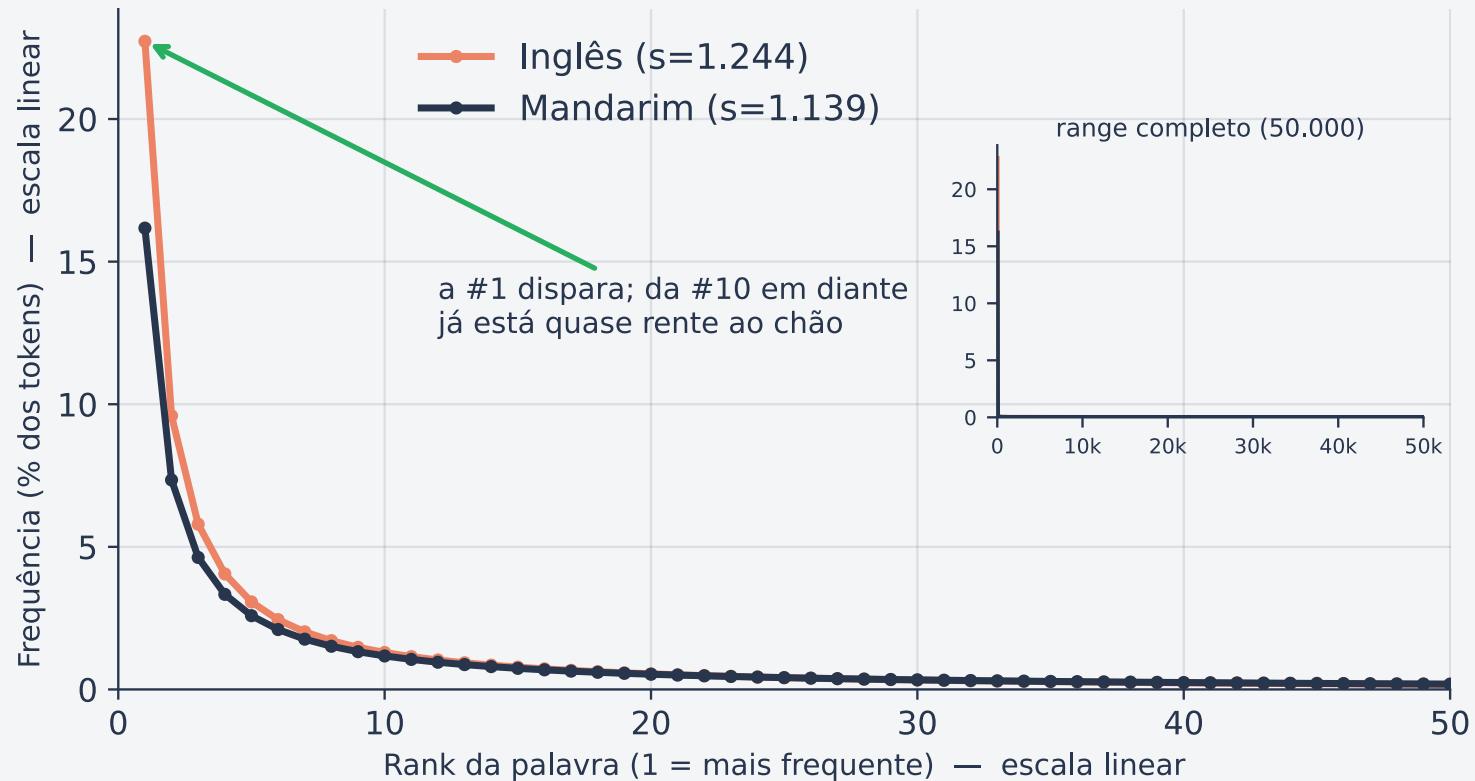
Quiz de Revisão da Aula 5

aqui

[Link para o Quizz da Aula 5](#)

Lei de Zipf - Distribuição da recorrência de palavras

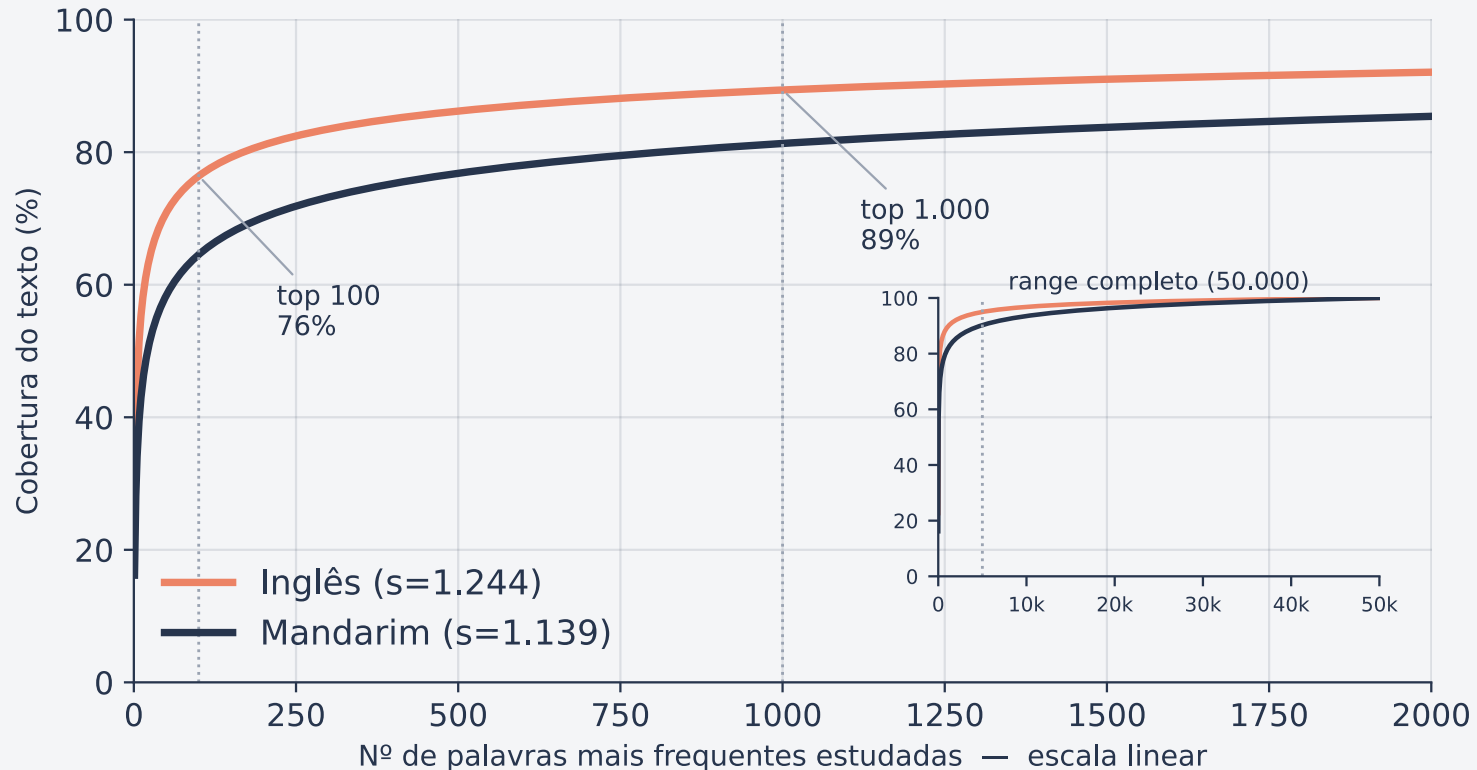
A mesma Lei de Zipf, sem o truque do log



Em escala linear a frequência **despenca** com o rank: a palavra mais comum vale milhares de vezes uma palavra rara. Estudar na ordem certa rende muito mais por palavra.

O ganho vem cedo

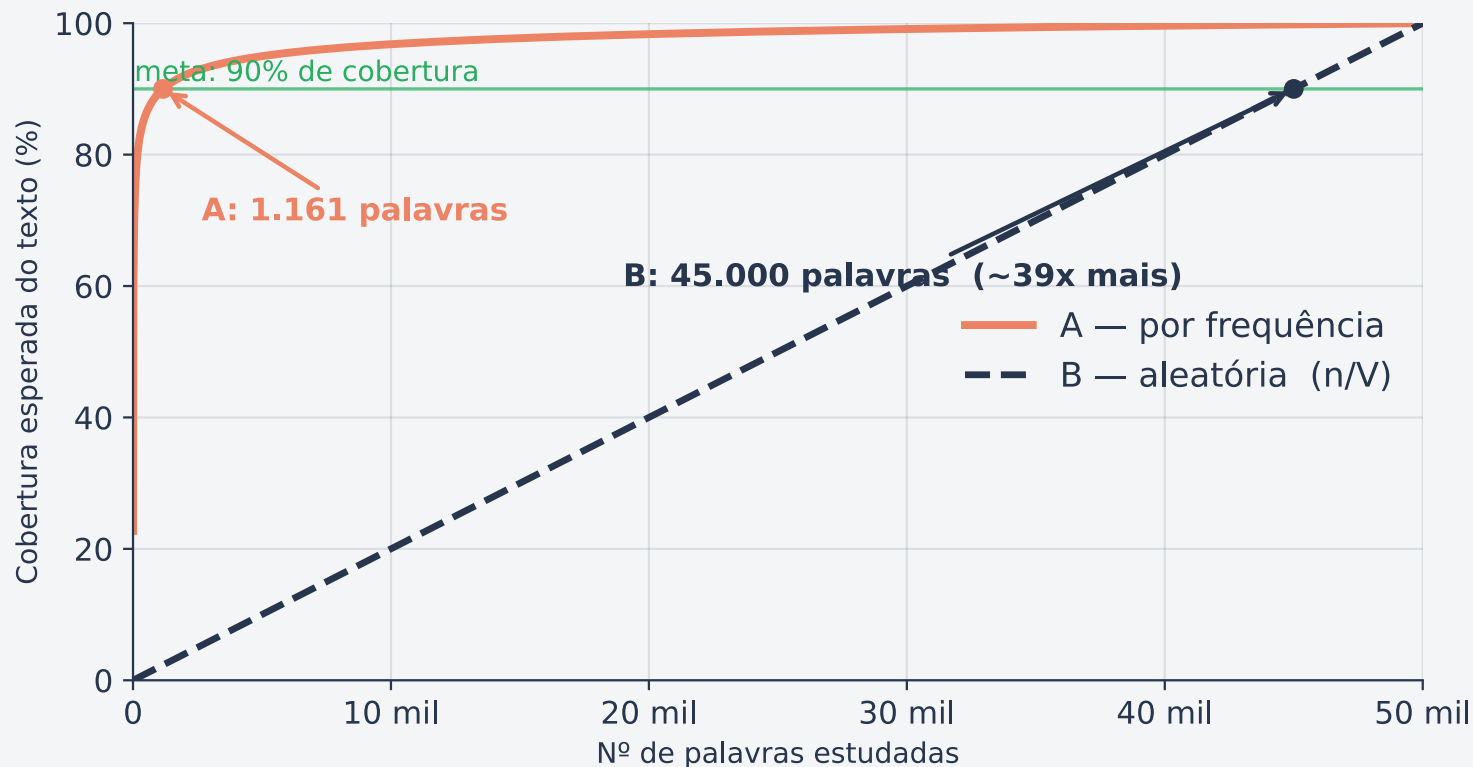
O grosso da compreensão vem das primeiras palavras



As **100** primeiras palavras já cobrem ~70–76% do texto; as **1.000** chegam a ~85–89%. Depois disso a curva achata — **retornos decrescentes**.

Estratégia: Frequência vs. Aleatório

Mesmo esforço, cobertura radicalmente diferente



Para a **mesma meta de 90%**, a ordem por frequência precisa de ~1.200 palavras; a ordem aleatória, ~45.000 — quase **40x mais esforço**.

Motivação

HTTP

Motivação

- Nos anos 1980 havia dezenas de variantes de Unix (BSD, System V, AIX, HP-UX, Solaris...), cada uma com pequenas diferenças.
- Um programa escrito para um **não compilava no outro**.
- POSIX (Portable Operating System Interface) definiu um contrato comum:
 - se o programa usar só o que a norma especifica, ele roda em qualquer SO compatível.

É o exemplo de "padronize a interface, não a implementação"

Abstração e Padronização

Na computação e sociedade digital

*A história pode ser lida como uma sucessão de camadas de **abstração** e regimes de **padronização**.*

Abstração - Conceito

- selecionar os aspectos relevantes de um sistema e ocultar, comprimir ou estabilizar os detalhes irrelevantes para um propósito.
 - Não é só simplificação; é **simplificação orientada a um propósito**.

open("arquivo.txt") — sem lidar com setores de disco, buffers, interrupções ou escalonamento.

Padronização - Conceito

- Criar, adotar e institucionalizar normas, especificações, protocolos e formatos
 - que reduzem ambiguidade e permitem **coordenação entre muitos atores**.

Tipo	Exemplo
Formal	ISO, IEEE, POSIX
Semiformal	W3C (web), OCI (container)
De fato	convenções de mercado/comunidade (QWERTY e REST)

Por que?

- Abstração
 - Reduz a complexidade local
- Padronização
 - Reduz a ambiguidade coletiva
- **Juntas, permitem**
 - Interoperabilidade, reúso, especialização, escala e inovação cumulativa.

Conceitos vizinhos (*?)

Conceito	O que é	Relação
Encapsulamento	Esconde estado/implementação atrás de uma interface	Implementa abstrações
Modularidade	Divide o sistema em partes independentes	Permite evolução paralela
Interface	Fronteira contratual entre componentes	Torna a abstração usável e padronizável
Interoperabilidade	Sistemas distintos trabalham juntos	Depende de padrões e semântica comum
Protocolo	Regras de interação entre entidades	Padroniza comunicação dinâmica
Plataforma	Base sobre a qual terceiros constroem	Combina abstração + padrão + governança

Linha do tempo dos marcos

Ano	Marco	O que abstraiu / padronizou
1968	Sistema THE (Dijkstra)	Camadas hierárquicas no SO
1970	Modelo relacional (Codd)	Independência lógica dos dados
1972	Parnas	Ocultamento de informação
1974–78	TCP/IP (Cerf & Kahn)	Interconexão de redes heterogêneas
1980s	POSIX	Interface comum de SO
1991	Unicode	Representação textual global
1994	W3C	Web como plataforma aberta
2000	REST (Fielding)	Estilo arquitetural de APIs
2011	NIST SP 800-145	Vocabulário comum de cloud
2015	OCI	Containers portáteis
2025	MCP / A2A	Protocolos da computação agentiva

Estudos de caso clássicos

- **Unix / POSIX** — abstrações simples e *composáveis* (arquivos, processos, pipes). *Abstrações simples + interfaces estáveis = vida longa.* 50" até 3'30"
- **TCP/IP** — padrão mínimo e poderoso: especifica só o suficiente. *Diversidade abaixo, inovação acima.*
- **Modelo relacional / SQL** — independência de dados. *Abstrações declarativas transferem complexidade do usuário para o sistema.* 1'35" até 3'57"
- **Web (HTTP/HTML/URL)** — padrões abertos + baixo custo de entrada = inovação descentralizada (mas capturável por intermediários).
- **Cloud + containers** — quando a infraestrutura vira API, a arquitetura técnica e o modelo organizacional mudam juntos.

Aprofundando: a pilha TCP/IP

O padrão mínimo que conectou redes heterogêneas e virou a espinha dorsal da internet

[Video Introdução](#)

TCP/IP — a ideia central: camadas

- O problema (anos 1970): havia **muitas redes físicas incompatíveis** (Ethernet, rádio, satélite, linhas telefônicas).
 - Como fazê-las conversar **sem exigir que todas fossem iguais**?
- A solução de Cerf & Kahn: **dividir a comunicação em camadas**,
 - cada uma com uma responsabilidade isolada, conversando apenas com a camada vizinha.

Cada camada resolve um problema e esconde os detalhes da camada de baixo da camada de cima. É a abstração e o encapsulamento aplicados à comunicação em rede.

As 4 camadas da pilha

Camada	Função	Pergunta que responde	Exemplos
Aplicação	Serviços usados pelos programas	<i>O que os dados significam?</i>	HTTP, DNS, SMTP
Transporte	Entrega fim-a-fim entre programas	Entregar a <i>quem</i> , com ou sem garantia?	TCP, UDP
Internet (Rede)	Endereçamento e roteamento entre redes	<i>Por onde</i> o pacote vai?	IP
Enlace / Acesso	Transmissão no meio físico concreto	Como mandar <i>bits</i> neste cabo/rádio?	Ethernet, Wi-Fi

TCP = confiabilidade (reenvia o que se perde, ordena).
IP = endereço + rota (mas não garante entrega). A dupla divide o trabalho.

Como um dado viaja: encapsulamento

Ao **enviar**, cada camada embrulha os dados da camada de cima com seu próprio cabeçalho — como envelopes dentro de envelopes:

```
dados → [TCP | dados] → [IP | TCP | dados] →  
[Ethernet | IP | TCP | dados]
```

No **destino**, cada camada abre só o seu envelope e entrega o conteúdo para a camada de cima (desencapsulamento).

- Cada camada **lê apenas o seu cabeçalho** e ignora o resto.
- Uma camada pode mudar sem afetar as outras
 - trocar Wi-Fi por cabo **não muda nada** no TCP ou no HTTP.

Por que importa: a "cintura fina"

A pilha tem formato de **ampulheta**: muita diversidade embaixo (mil tecnologias de rede) e em cima (mil aplicações), mas **um único ponto estreito no meio — o IP**.

IP padronizado: *Padronizar só a camada do meio foi suficiente para unir o mundo*

- **Padrão mínimo, efeito máximo** — especificar de menos no ponto certo foi a força, não a fraqueza.

DNS — a "lista telefônica" da internet

- Humanos lembram **nomes** (gov.br);
- máquinas roteiam por **números IP** (161.148.x.x).
- O **DNS** (*Domain Name System*) traduz um no outro.

Etapa	O que acontece
Você digita gov.br	o navegador precisa do IP correspondente
O resolver consulta a hierarquia	Nó raiz consulta zona (ex .br)
Volta o IP	aí sim o HTTP sobre TCP/IP entra em ação

Abstração pura: o nome é estável, mas o IP por trás pode mudar **sem o usuário perceber** — desacopla identidade de localização.

HTTP — o protocolo da web

- Sobre o TCP/IP, a web precisava de um **idioma comum** para clientes (navegadores) e servidores conversarem.
- Esse idioma é o **HTTP** (*HyperText Transfer Protocol*).
- Modelo simples de **requisição → resposta**:
 - o cliente *pede*, o servidor *responde*.

Método	O que pede
GET	Buscar um recurso
POST	Enviar/criar dados

- É **sem estado** (*stateless*): cada requisição é independente
- o que dá escala

Exemplo real: uma chamada HTTP

```
curl -i -X GET https://example.com
```

A **URL** diz o que queremos e onde:

https:// + **example.com** + **/**
protocolo servidor/host caminho do recurso

```
HTTP/2 200
date: Tue, 09 Jun 2026 01:32:00 GMT
content-type: text/html
server: cloudflare
last-modified: Thu, 04 Jun 2026 22:59:45 GMT
allow: GET, HEAD
accept-ranges: bytes
age: 1403
cf-cache-status: HIT
cf-ray: a08c61495e25336d-MIA

<!doctype html><html lang="en"><head><title>Example Domain</title>...</html>
```

200 = deu certo · content-type: text/html = "o que vem a seguir é HTML".

Aprofundando: APIs REST

Como sistemas conversam entre si usando HTTP como base

O que é REST (Representational State Transfer) - Vídeo

- É um estilo arquitetural
 - um **conjunto de convenções** para projetar APIs sobre o HTTP.
- A ideia central: expor tudo como **recursos** (substantivos) identificados por **URLs**,
 - manipulados pelos **verbos** que o HTTP já oferece.

*Em vez de inventar comandos novos (/buscarProcesso, /criarProcesso...), REST **reaproveita o que o HTTP já padronizou**: GET, POST, PUT, DELETE.*

- É a fusão **abstração + padronização** do início da aula: o consumidor não conhece a implementação (abstração), mas conhece o contrato (padronização).

Recursos e URLs — substantivos, não verbos

Em REST, **cada coisa é um recurso** com um endereço próprio. A URL identifica o *quê*; o verbo HTTP diz o *que fazer*.

URL	O que representa
/processos	A coleção de todos os processos
/processos/2026-0042	Um processo específico
/processos/2026-0042/documentos	Os documentos daquele processo

✓ `GET /processos/2026-0042` ✗ `GET /buscarProcessoPorId?id=...`

- URLs são **hierárquicas e previsíveis**
 - quem conhece o padrão "adivinha" os outros endpoints.

Verbos HTTP = operações (CRUD)

REST mapeia os 4 verbos HTTP nas 4 operações básicas de dados (CRUD):

Verbo	Operação	Exemplo
GET	Read — ler	GET /processos/2026-0042
POST	Create — criar	POST /processos
PUT / PATCH	Update — atualizar	PUT /processos/2026-0042
DELETE	Delete — remover	DELETE /processos/2026-0042

O verbo decide a operação. Uma URL, várias ações.

Exemplo prático: GET (ler)

Pedir os dados de um processo. A resposta vem em **JSON** (formato de dados padronizado):

```
curl https://api.exemplo.gov.br/v1/processos/2026-0042
```

```
HTTP/2 200 OK  
content-type: application/json
```

```
{  
  "id": "2026-0042",  
  "assunto": "Licença ambiental",  
  "status": "em_analise",  
  "orgao": "SEMA",  
  "atualizado_em": "2026-06-08"  
}
```

*200 OK + a **representação** do recurso em JSON.*

Uma requisição numa API real

- API de dados abertos do **IBGE**
 - peça a unidade federativa 53 e a resposta vem em **JSON**

```
curl https://servicodados.ibge.gov.br/api/v1/localidades/estados/53
```

```
HTTP/1.1 200 OK  
content-type: application/json; charset=utf-8
```

```
{  
  "id": 53,  
  "sigla": "DF",  
  "nome": "Distrito Federal",  
  "regiao": { "id": 5, "sigla": "CO", "nome": "Centro-Oeste" }  
}
```

Exemplo prático: POST (criar)

Criar um novo processo: enviamos o corpo em JSON com POST:

```
curl -X POST https://api.exemplo.gov.br/v1/processos \
-H "Content-Type: application/json" \
-d '{"assunto": "Licença ambiental", "orgao": "SEMA"}'
```

```
HTTP/2 201 Created
Location: /v1/processos/2026-0043
```

*201 Created (não 200) + cabeçalho Location apontando para o **novo recurso** recém-criado. O servidor gerou o *id* e diz onde encontrá-lo.*

POST na API JSONPlaceholder

```
curl -i -X POST https://jsonplaceholder.typicode.com/posts \  
  -H "Content-Type: application/json" \  
  -d '{"assunto": "Licença ambiental", "orgao": "SEMA"}'
```

```
HTTP/2 201 Created  
content-type: application/json; charset=utf-8  
location: https://jsonplaceholder.typicode.com/posts/101
```

```
{  
  "assunto": "Licença ambiental",  
  "orgao": "SEMA",  
  "id": 101  
}
```

*JSON enviado, o servidor respondeu 201, devolveu o recurso com o **id que ele gerou** e aponta o Location.*

Códigos de status da resposta da requisição HTTP

- A resposta sempre traz um **código** que diz, de forma padronizada, o que aconteceu
 - o cliente trata o erro sem "adivinhar":

Faixa	Significado	Exemplos
2xx	Sucesso	200 OK, 201 Created, 204 No Content
4xx	Erro do cliente	400 (pedido inválido), 401 (não autenticado), 403 (sem permissão), 404 (não existe)
5xx	Erro do servidor	500 (erro interno), 503 (indisponível)

A faixa já diz **de quem é a culpa**: 4xx = você errou o pedido · 5xx = o servidor falhou.

MCP: abstração e padronização na IA agêntica

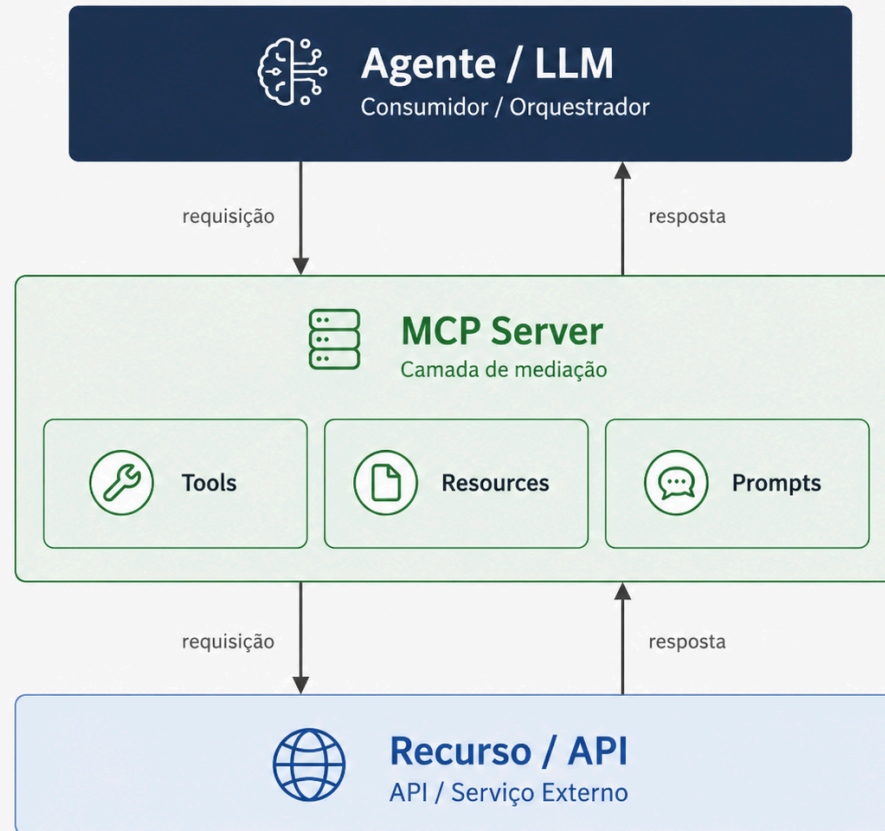
Model Context Protocol — o "contrato" que conecta agentes de IA a dados, ferramentas e sistemas reais

Dando mãos, braços e pernas para os LLMs

*Modelos de IA só geram valor público quando conseguem acessar **dados, ferramentas e executar operações em sistemas reais**.*

- LLMs sozinhos **respondem**;
 - agentes conectados podem **agir**.
- Sem padrão, cada integração vira um **conector artesanal**.
 - Cada agente produzindo um código que outro agente já produziu
- O **MCP** surge para reduzir essa fragmentação.

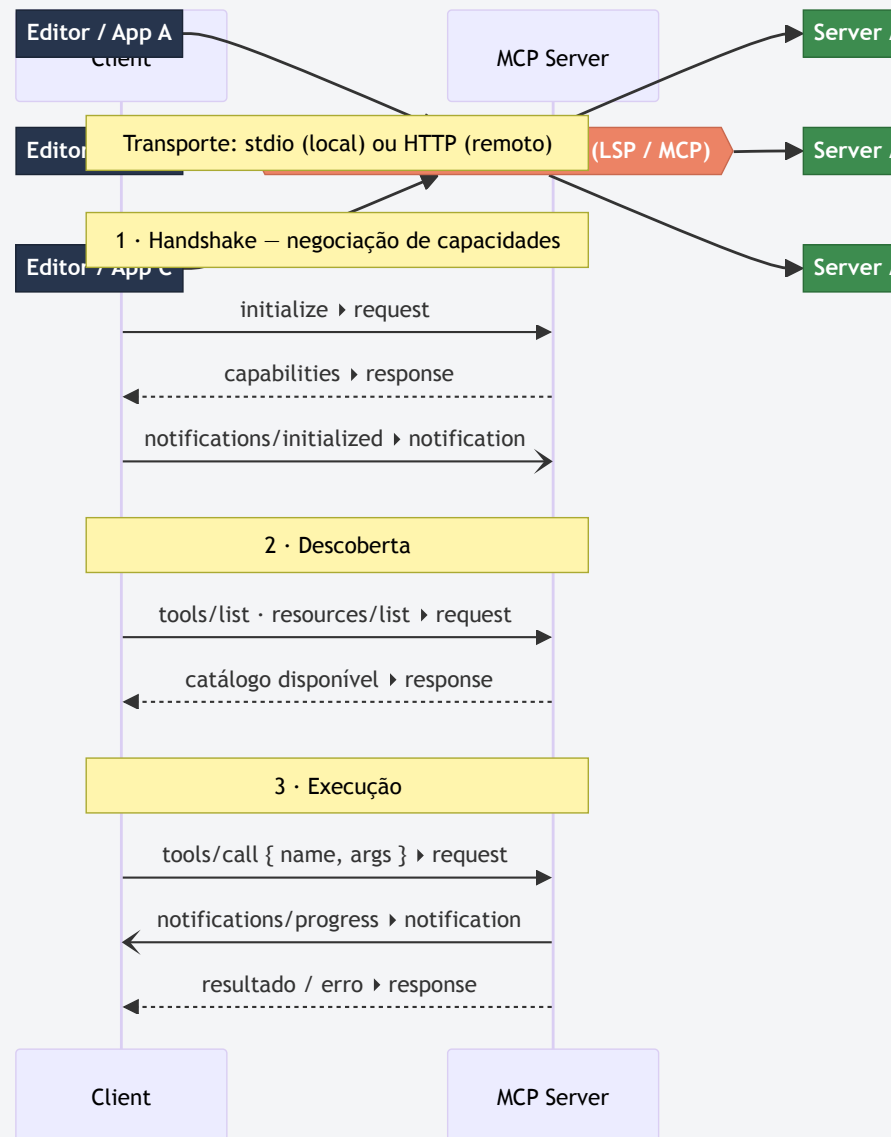
MCP em 3 camadas



o LLM nunca fala direto com o recurso — o MCP é a camada do meio que padroniza o acesso (1 protocolo para N recursos)

Por que o LSP inspirou o MCP: de $M \times N$ para $M + N$

- **Sem padrão:** M clientes $\times$ N serviços = $M \times N$ integrações artesanais.
- **Com um protocolo comum:** o custo cai para $M + N$
 - escreve o server **uma vez**, roda em **qualquer** cliente que fale o protocolo.



LSP (2016)

Cliente

Editor (VS Code, Vim...)

Server

language server (pyright...)

MCP (2024)

App de IA (Cursor, Claude...)

MCP server

O problema que o MCP resolve - Inspirado no LSP

Antes do MCP, conectar LLMs a ferramentas era **fragmentado**: cada integração era artesanal, platform-specific e unidirecional.

- **Function calling** (OpenAI, 2023), **plugins**, **frameworks** (LangChain, LlamaIndex)
 - cada um com autenticação, schema e lógica próprios — redundância e baixa manutenibilidade.
- **MCP** (Anthropic, fim de 2024): um **protocolo aberto** que padroniza
 - *definição, descoberta e invocação* de ferramentas externas.

Três inovações sobre o function calling tradicional:

- **Padrão baseado em protocolo** — desacopla a *implementação* da ferramenta do seu *uso*.
- **Descoberta dinâmica + negociação de schema** — o client lista e invoca tools em runtime, sem hardcoded.
- **Canais bidirecionais** — não só model→tool,
 - mas também eventos e notificações tool→host.

A arquitetura do MCP

- **Host:** aplicação de IA que coordena a experiência (Ex.: VS Code).
- **Client:** conector dentro do host.
- **Server:** sistema que oferece contexto ou capacidades.
- Um host conecta-se a **vários servers**;
 - servers podem ser **locais ou remotos**.

Primitivas: tools, resources e prompts

Primitiva	O que é	Exemplo no setor público
Tools	Funções executáveis	consultar processo, gerar documento, abrir chamado
Resources	Dados e contexto	arquivos, registros, bases, metadados
Prompts	Templates/fluxos reutilizáveis	roteiros de interação controlados

Transporte: como client e server se conectam

- **stdio** — o client **lança o server como subprocesso** e troca mensagens por stdin/stdout.
 - Tudo na mesma máquina.
- **Streamable HTTP** — o server roda como **processo independente**, usa **HTTP POST/GET**
 - para streaming, notificações e mensagens server→client.

Fluxo de mensagens: como client e server conversam

Três tipos de mensagem (JSON-RPC 2.0): **Request** → tem *id*, espera resposta | **Response** ← mesmo *id*, devolve *result/error* | **Notification** → sem *id*, não espera resposta

stdio vs. Streamable HTTP

Transporte	Melhor para	Vantagens	Trade-offs
stdio	Dev local, IDEs, desktop	Simples, baixa latência local, fácil de empacotar	Executa processo local; risco de <i>command execution</i> ; difícil escalar
Streamable HTTP	Produção, SaaS, times distribuídos	Deploy independente, autenticação HTTP, multi-cliente	Exige auth, CORS/Origin, sessão, observabilidade

stdio = *pipe na própria máquina*

Streamable HTTP = *serviço de rede com tudo que rede exige.*

A abstração também pode esconder perigo

Agentes conectados a ferramentas exigem
governança forte.

- **Prompt injection** direta e indireta.
- Execução indevida de ferramentas e **excesso de permissão**.
- **Vazamento de dados** e dificuldade de auditoria.
- Dependência de fornecedores.
- Confusão entre **recomendação** e **decisão**.

A própria *especificação* exige consentimento, privacidade e controles de autorização; a *NSA* publicou *orientação de segurança* sobre MCP.

O artigo: MCP em perspectiva

Hou, Zhao, Wang & Wang (HUST, 2025) — "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions"

[arXiv:2503.23278](https://arxiv.org/abs/2503.23278)

Ciclo de vida do MCP server (4 fases · 16 atividades)

Fase	Atividades-chave
Creation	Definição de <i>metadata</i> · Declaração de <i>capabilities</i> · Implementação do código · Definição de <i>slash commands</i>
Deployment	Release do server · <i>Installer</i> · Setup de ambiente · Registro de tools
Operation	Análise de <i>intent</i> · Acesso a recursos externos · Invocação de tools · Gestão de sessão
Maintenance	Controle de versão · Mudança de configuração · Auditoria de acesso · Auditoria de logs

O ciclo de vida é o eixo do artigo: cada **ameaça** é mapeada à fase em que **se origina** — não só onde se manifesta (ex.: *tool poisoning* nasce na *creation*, dispara na *operation*).

Ecossistema: adoção acelerada, maturidade desigual

- **Players & frameworks:** Anthropic (Claude), OpenAI , Cursor, Baidu, Microsoft Copilot, IDEs, fintech (Stripe).
- **Servers comunitários:** diretórios como **MCPWorld (~26 mil)**, **MCP.so (~16 mil)**,
 - mas **sem verificação formal de identidade/segurança**.
 - Amostra de 300 servers no MCP.so: ~10% nem eram MCP; ~6% inativos → contagens **infladas**, qualidade variável.
- **Casos validados: OpenAI** (integração unificada de tools), **Cursor** (assistentes de código)

Segurança: 4 tipos de atacante × 16 ameaças

Atacante (fase de origem)	Ameaças representativas
Malicious Developer (creation)	Namespace typosquatting · Tool name conflict · Preference manipulation · Tool poisoning · Rug pulls · Cross-server shadowing · Command injection
External Attacker (deploy/operation)	Installer spoofing · Indirect prompt injection
Malicious User (operation)	Credential theft · Sandbox escape · Tool chaining abuse · Unauthorized access
Security Flaws (maintenance)	Vulnerable versions · Privilege persistence · Configuration drift

Alguns dos ataques

- **Tool poisoning** — lógica maliciosa escondida atrás de interface legítima; o `add(a,b)` correto também vaza uma informação sensível (Ex.: chave SSH).
- **Rug pull** — server benigno ganha confiança e, em *update* posterior, injeta payload malicioso (*temporal stealth*).
- **Typosquatting** — `mcp-github` se passa por `github-mcp`; o agente escolhe só pelo nome/descrição textual.

*Raiz comum: seleção de server/tool feita por **texto não verificado**, automatizada por agentes — sem assinatura nem proveniência.*

Exercícios — tutoriais no GitHub

Leia (rapidamente) todo o tutorial antes de começar

- **Exercício 4.1 — API REST da TODO list (POST/GET/PUT):** [tutorial_4.1.md](#)
- **Exercício 4.2 — MCP server local que consome a API do 4.1:** [tutorial_4.2.md](#)

*Avaliados por **execução real** via autograde validar 4.1 / 4.2.*

